OOP Concepts

7.1 Inheritance

7.2 Polymorphism

Intended Learning Outcomes

After completing the lesson, students should be able to:

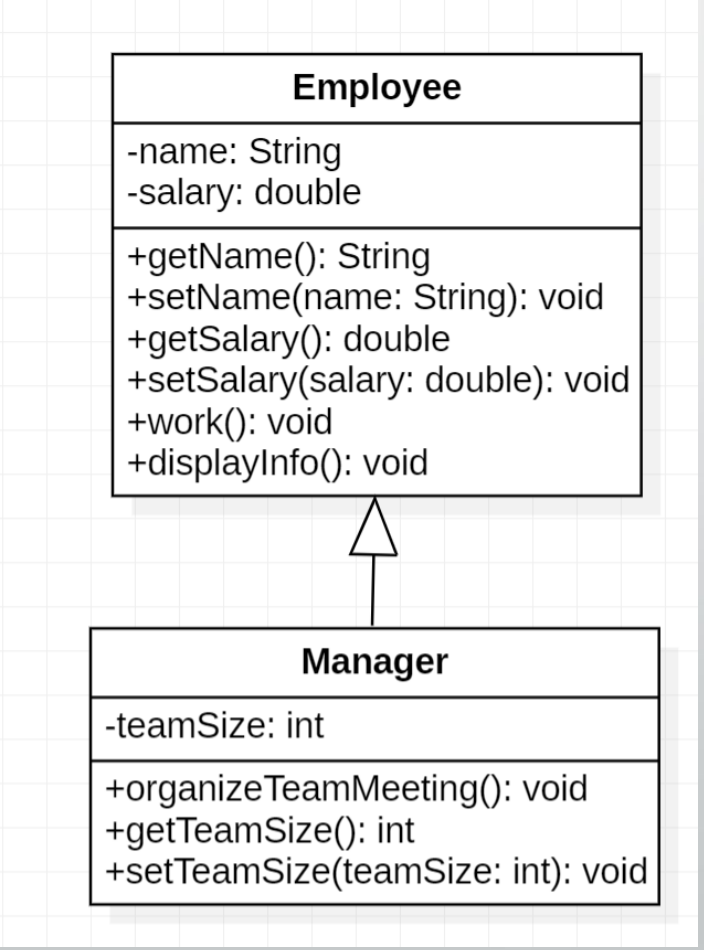
- Describe the concepts of inheritance and polymorphism
- Design Java classes that inherit from other classes
- Utilize polymorphism by method overriding, method overloading

Inheritance

- fundamental concept in OOP
  - allows a class to inherit properties and methods from another class.
- a powerful tool for creating organized and maintainable code
- avoid creating overly complicated class hierarchies
  - difficult to understand and maintain.

Parent and Child classes

- Parent class (super class)
  - class from which the properties are inherited
  - e.g. Employee
- Child class (subclass)
  - class that inherits the properties
  - e.g. Manager



Inheritance in Java

- Syntax:

```
class [ChildClass] extends [ParentClass]
```

- ChildClass would inherit
  - all fields and methods
  - from the ParentClass.
- Example:

```
class Manager extends Employee
```

"is-a" relationship

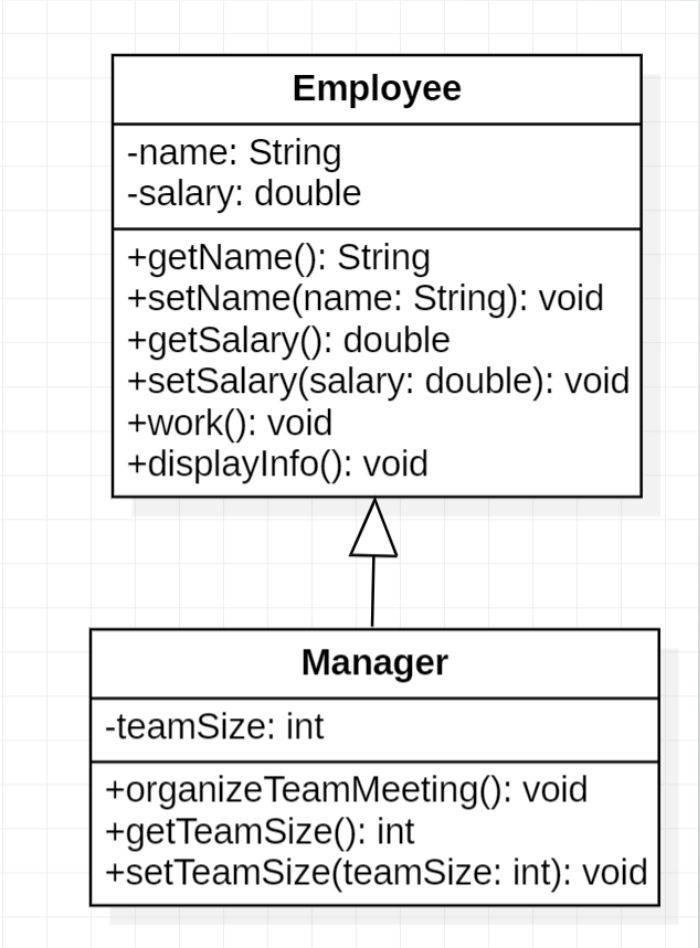
- Inheritance is used to create an "is-a" relationship
- Example
  - Manager is an Employee.
- Child class (specialized object) has:
  - all the characteristics (methods, fields) of the general object (Parent class)
  - + additional characteristics that make it special.

Exercise – Brainstorm inheritance

- Brainstorm inheritance examples in
  - Animal World
  - Vehicles
  - Electrical Appliance

Inheritance Example - Manager

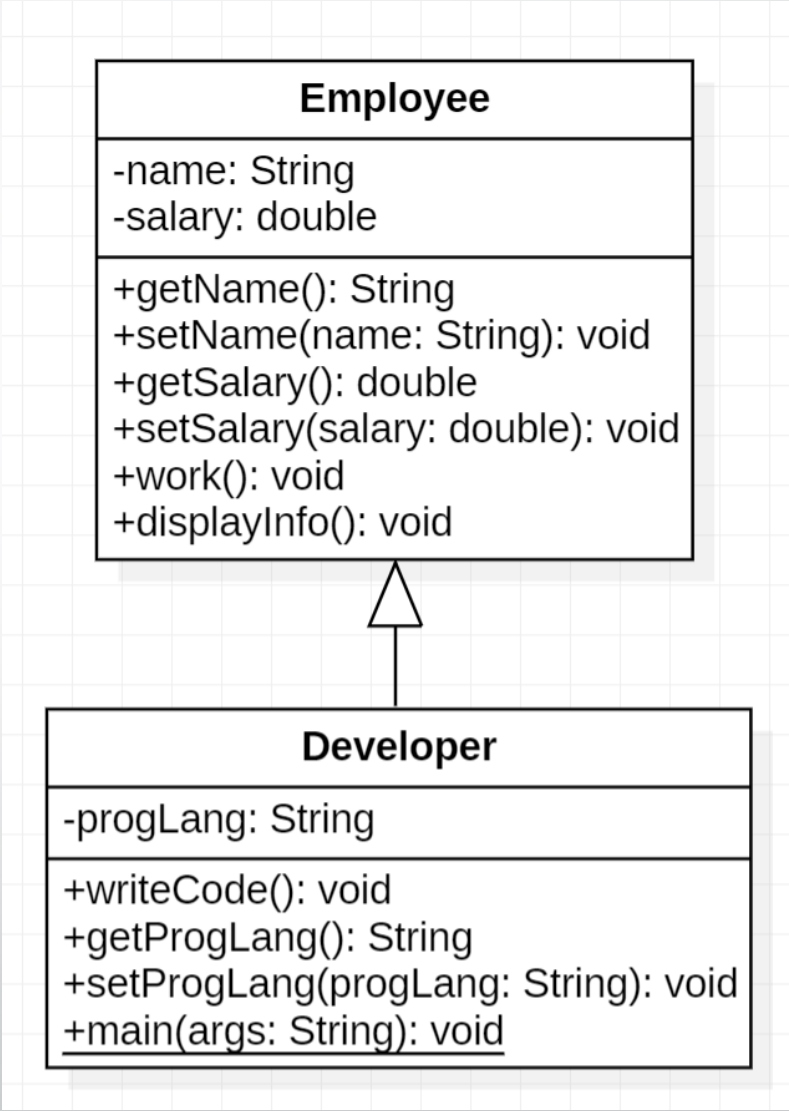
```java
public class InheritanceExample{
    public static void main(String[] args) {
        Manager manager = new Manager();
        manager.setName("John Doe");
        manager.setSalary(5000.00);
        manager.setTeamSize(10);
        manager.displayInfo();
      // Inherited from Employee class
        manager.organizeTeamMeeting();
        // Specific to Manager class
    }
}
```



See: InheritanceExample.java, Employee.java, Manager.java

Exercise - Developer

- Write a Developer class that inherits from Employee
- A private String field called progLang to store the programming language used by the developer.
- Plain getter and setter method.
- A public method called writeCode() that simply prints out `"Developer write code in [progLang]"` where [progLang] is the actual programming language.
- Write a main() function to demonstrate the methods.
  - Follow previous example



Key points about inheritance

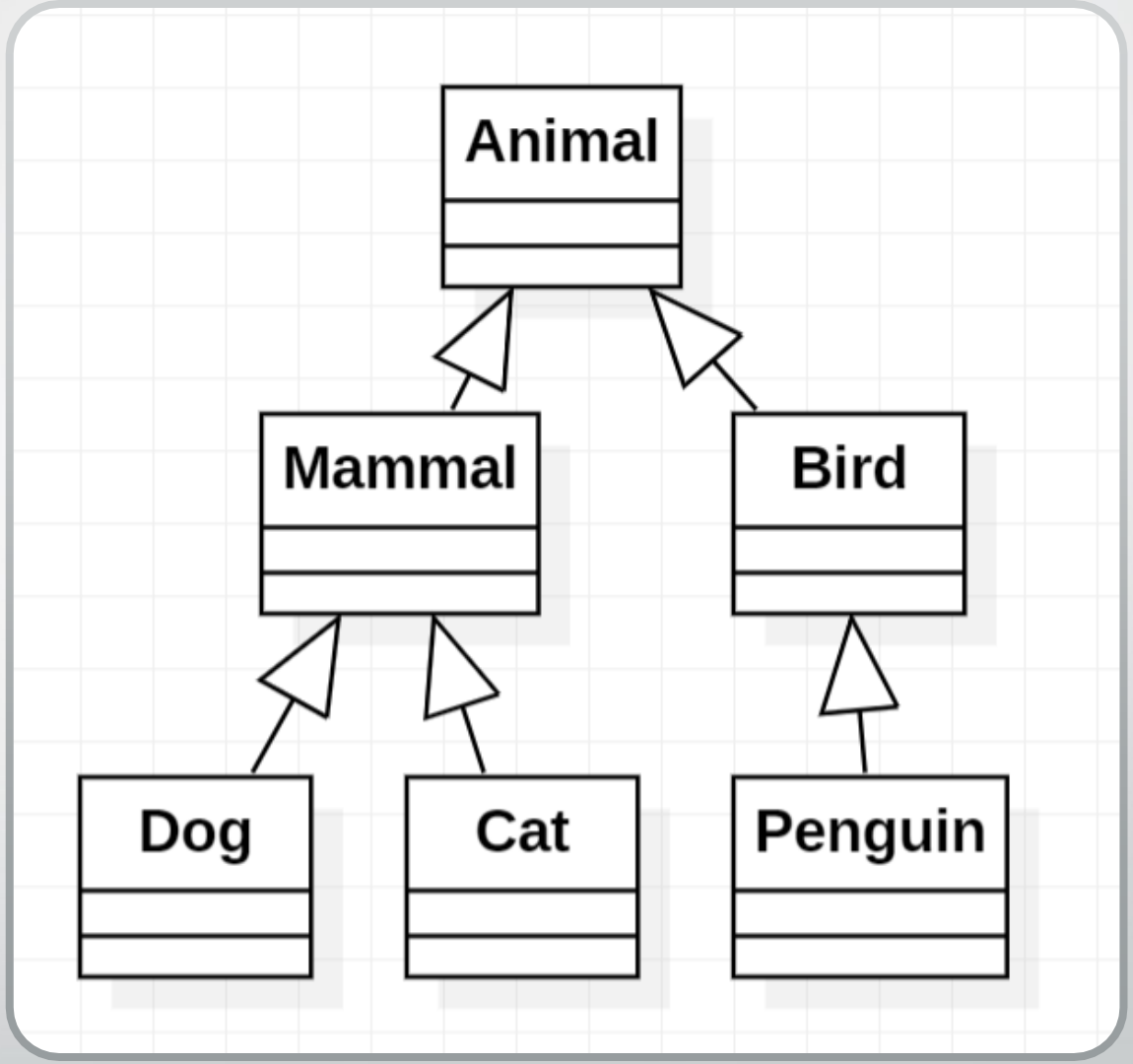
- Code Reusability
- Hierarchy
- Constructors and Destructors
- Access Modifiers
- Method Overriding
- Polymorphism
- Multiple Inheritance

Code Reusability

- Inheritance supports the concept of "don't repeat yourself" (DRY).
- create a new class that reuses, extends, or modifies the behavior of an existing class.
- reuse of existing code
  - saves time and effort
- Example
  - `manager.displayInfo();`

Hierarchy

- Inheritance creates a hierarchy within classes.
- Subclass
  - Specific implementation

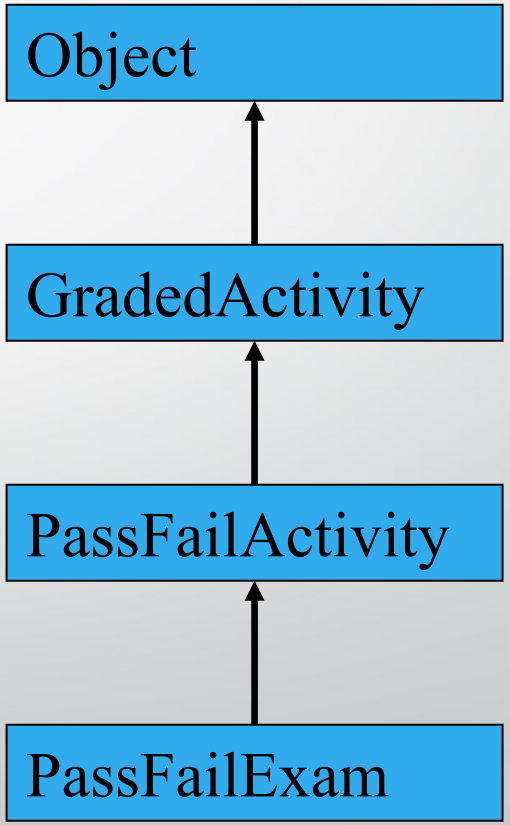


Chains of Inheritance

- A superclass can also be derived from another class.

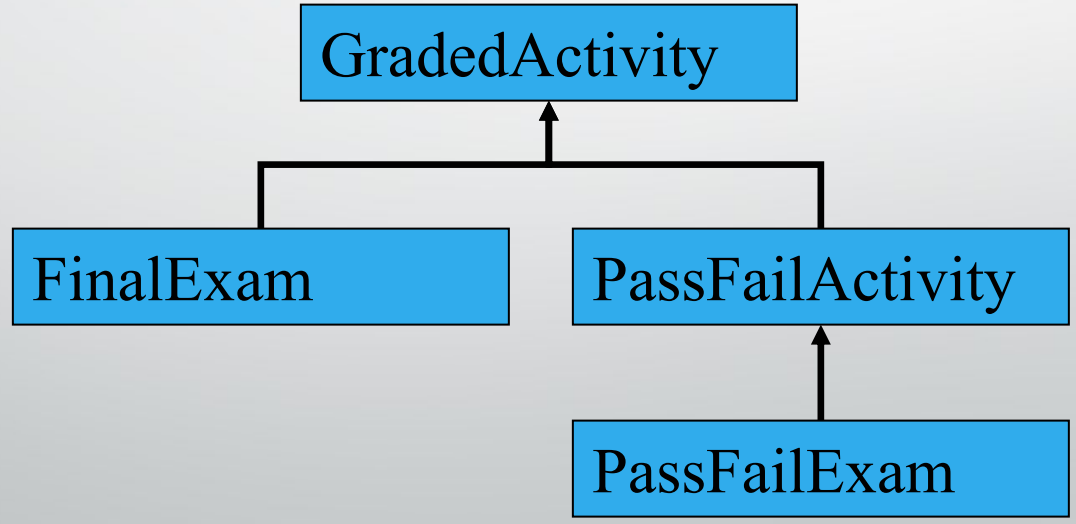
Example:

GradedActivity.java
PassFailActivity.java
PassFailExam.java
PassFailExamDemo.java



Chains of Inheritance

- Classes often are depicted graphically in a *class hierarchy*.
- A class hierarchy shows the inheritance relationships between classes.



The `Object` Class

```
public class MyClass
{
    // This class is derived from Object.
}
```

- All Java classes are directly or indirectly derived from a class named `Object`.
  - `Object` is in the `java.lang` package.
- Any class that does not specify the `extends` keyword
  - automatically derived from the `Object` class.
- Ultimately, every class is derived from the `Object` class.

The `Object` Class

- Every class inherits the `Object` class's members.
- In the `Object` class,
  - `toString` method
    - returns a string containing the object's class name and a hash of its memory address.
  - `equals` method
    - accepts the address of an object as its argument and
    - returns true if it is the same as the calling object's address.
- Example: ObjectMethods.java

Constructors and Destructors

- Fields of parent class
  - Initialized in Parent's constructor
  - Destroyed in Parent's destructor (in C++)
- In Java, parent's constructor can be called using `super`(…)
- Example:

```
Manager(String name, double salary, int teamSize){
    super(name, salary);
    this.teamSize = teamSize;
}
```

Object Destruction in Java

- No explicit destructor (unlike `~` in C++)
  - `finalize()` exists but is unreliable and should be avoided.
- For resource cleanup (e.g., closing files), use
  - `try-with-resources` or
  - `finally` blocks (in Topic of Exception Handling)

Garbage Collection in Java

- Automatic memory management
  - reclaims memory from unreachable objects.
- Unreachable objects are garbage collected.
  - Reachable objects are accessible;
- JVM automatically handles garbage collection
- `System.gc()` is a suggestion,
  - not a guarantee; avoid direct use.

Access Modifier

Access Modifiers

- Controls visibility & accessibility of fields & methods

Access Modifier	within class	within package	outside package by **subclass only**	outside package
`private`	Y	N	N	N
`[default]`	Y	Y	N	N
`protected`	Y	Y	Y	N
`public`	Y	Y	Y	Y

Reference: https://www.javatpoint.com/access-modifiers

Package in Java

- Code Organization
  - provides a hierarchical structure for organizing and managing Java classes.
  - as a single unit
- Namespace management
  - helps avoid classes naming conflicts
  - Companies use reversed Internet domain name to begin their package names
- Encapsulation - package-level visibility

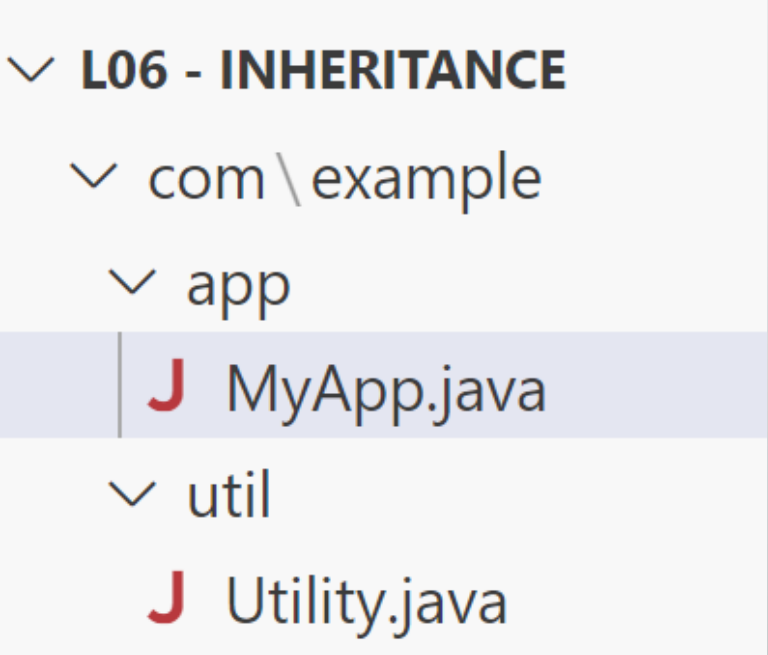
Naming Convention - https://docs.oracle.com/javase/tutorial/java/package/namingpkgs.html

Create and import package

- Packages are represented by directories in the file system.
  - Each package corresponds to a directory,
  - Java source files (.java) and compiled bytecode files (.class) are stored within their respective package directories.
- Name a package
  - `package com.example.app;`
- Import a class in a package
  - `import com.example.util.Utility;`

Package Example - com.example.app.MyApp.java

```java
package com.example.app;
import com.example.util.Utility;
public class MyApp {
    public static void main(String[] args) {
        com.example.util.Utility util =
            new Utility();
        util.helper();
    }
}
```



Default = package

- If no access specifier for a class member is provided,
  - the class member is given ***package access*** by default.
- Any method in the same package
  - may access the member.

Protected Members

- `protected` members of class:
  - may be accessed by methods in a subclass, and
  - by methods in the same package as the class.
- Example:
  - GradedActivity2.java, FinalExam2.java, ProtectedDemo.java

Protected Members

- Using `protected` instead of `private` makes some tasks easier.
- unrestricted access to the protected member.
  - any derived class or fromthe same package,
- always better to make all fields `private` and
- then provide `public` methods for accessing those fields.

Method Overriding

Polymorphism in a class

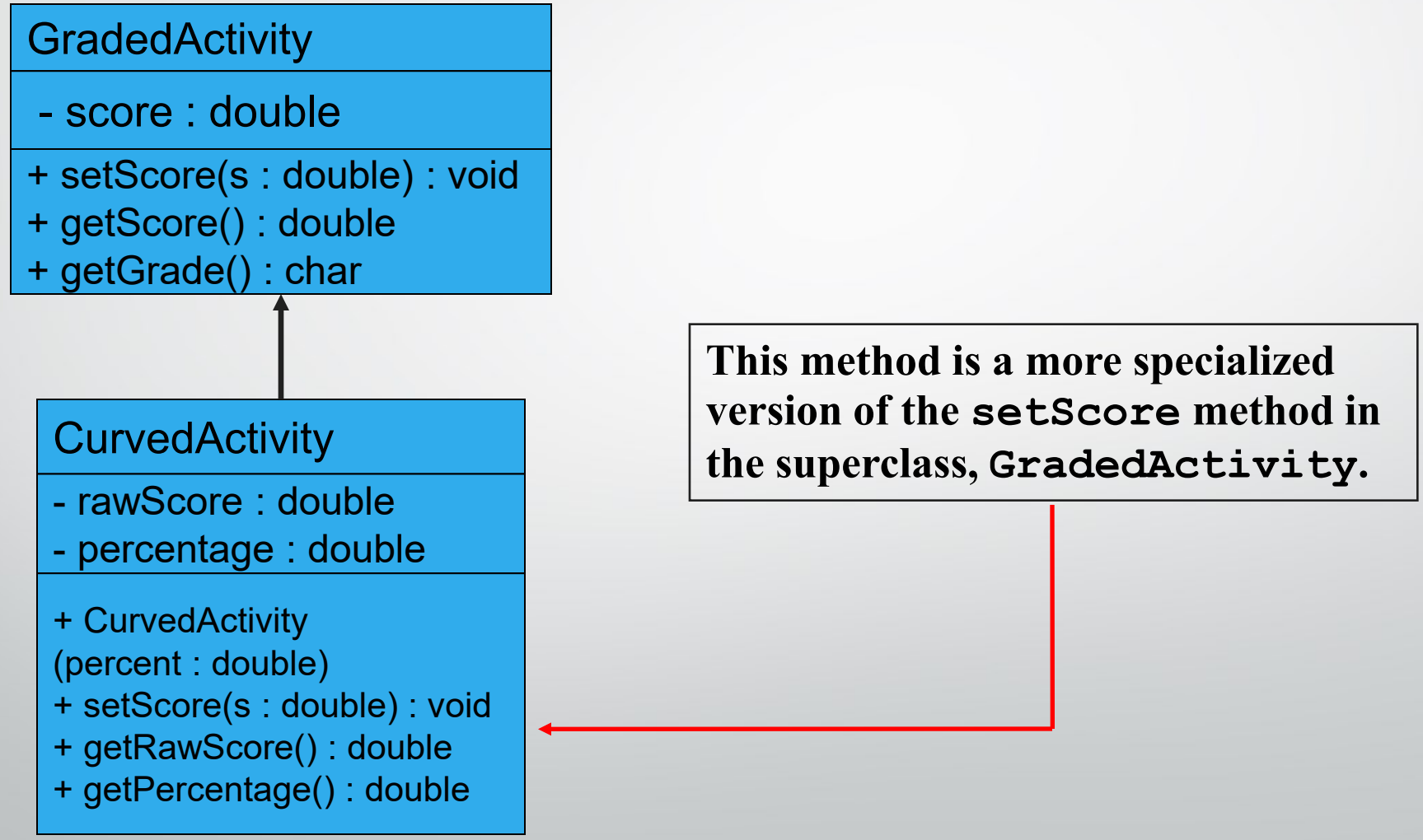
Method Overriding

- Subclasses can override methods from the parent class.
- even though a method is inherited from the parent,
  - a subclass can provide its specific implementation

Overriding Superclass Methods

- A subclass may have a method with the same signature as a superclass method.
- The subclass method overrides the superclass method.
- This is known as *method overriding*.
- Example:
  - GradedActivity.java, CurvedActivity.java, CurvedActivityDemo.java

Overriding Superclass Methods



Overriding Superclass Methods

- Recall that a method's *signature* consists of:
  - the method's name
  - the data types method's parameters in the order that they appear.
- A subclass method that overrides a superclass method must have the same signature as the superclass method.
- An object of the subclass invokes the subclass's version of the method, not the superclass's.

Overriding Superclass Methods

- An subclass method can call the overridden superclass method via the super keyword.

```
super.setScore(rawScore * percentage);
```

- There is a distinction between overloading a method and overriding a method.
- Overloading is when a method has the same name as one or more other methods, but with a different signature.
- When a method overrides another method, however, they both have the same signature.

Overriding Superclass Methods

- Both overloading and overriding can take place in an inheritance relationship.
- Overriding can only take place in an inheritance relationship.
- Example:
  - SuperClass3.java,
  - SubClass3.java,
  - ShowValueDemo.java

Preventing a Method from Being Overridden

```
public final void method()
```

- `final` modifier will prevent
  - the overriding of a superclass method in a subclass.
- If a subclass attempts to override a final method,
  - the compiler generates an error.
- ensures that a particular superclass method is used by subclasses
  - rather than a modified version of it.

Polymorphism

Polymorphism

- *polymorphism* means the ability to take many forms.
  - In Java, a reference variable is *polymorphic*
- use of a single type to represent objects of different types
  - if they are subclasses of the same parent class

Dynamic binding (Runtime polymorphism)

- method to be called is determined at runtime,
  - not compile time.
- vs early binding: method is resolved at compile time.
- Java performs *dynamic binding* (*late binding)*
- JVM determines at runtime which method to call,
  - depending on the type of object that the variable references.

Example - Polymorphism.java

- For example, a function that takes an `Animal` type
  - can operate on `Dog` or `Fish` objects.

```
Animal pet1 = new Animal();
pet1.breath();

Animal dolly = new Dog();
dolly.breath();
// Cannot dolly.bark() without casting

Animal nemo = new Fish();
nemo.breath();
Fish nemo2 = (Fish) nemo;
nemo2.swim(); // Fish has swim()
```

@Override Annotation

- **Purpose**: Indicates a method overrides a superclass method; enables compile-time checking.
- **How it Works**:
  - The compiler verifies the override;
  - errors if the method doesn't exist in the superclass or the signature doesn't match.
- **Benefits**: Improved code readability, reduced errors, increased maintainability.
- Best Practice: Always use `@Override` when overriding a superclass method.

Polymorphism and Dynamic Binding

- If the object of the subclass has overridden a method in the superclass:
  - If the variable makes a call to that method the subclass's version of the method will be run.

```
GradedActivity exam = new PassFailActivity(60);
exam.setScore(70);
System.out.println(exam.getGrade());
```

Polymorphism

- It is the object’s type, rather than the reference type, that determines which method is called.
- Example: Polymorphic.java
- cannot assign a superclass object to a subclass reference variable.
- Can cast a reference to it's original type.

Polymorphism Exercise - Cars

- Create a program that models different types of vehicles.
- **Vehicle** (parent class):
  - Create a `start()` method that prints "Vehicle starting...".
  - Create a method called `drive()` that prints a generic message like "Driving a vehicle."
- **Car** (child class of `Vehicle`):
  - Inherit from Vehicle class
  - Override the `start()` method to print "Car starting...".
  - Add a method called `drift()` that prints "Car drifting!".

Multiple Inheritance

- Inheritance from multiple classes
  - Allowed in C++, May Introduce complexity
- Not supported in Java
- Java offer `interface` as an alternative - Next session

Polymorphism - Method Overloading

Method Overloading

- allows a class to have multiple methods
- with the **same name** but **different parameters**.
- to create methods that perform similar operations
  - but with different input parameters.
- compiler determines the appropriate version of the method to call
  - based on the number and types of arguments passed.

Method Overload (Java)

- Same method name
- Different parameters:
  - must have different parameter lists.
  - can differ in terms of their number, types, or order.
- Return type: **not** considered when overloading methods.
- cannot be overload:
  - same name, same parameter types but different return types

Method Overload Example (Calculator.java)

```java
public class Calculator {
    public int add(int a, int b) { return a + b; }

    public double add(double a, double b) { return a + b; }

    public int add(int a, int b, int c) { return a + b + c; }
    public static void main(String[] args){
        Calculator calculator = new Calculator();
        int result1 = calculator.add(2, 3);             // Invokes the first add method
        double result2 = calculator.add(2.5, 3.7);      // Invokes the second add method
        int result3 = calculator.add(2, 3, 4);          // Invokes the third add method
        System.out.println(result1 +", " + result2 + ", " + result3);
    }
} // class
```

Default value by Method Overloading

- In Java method arguments
  - do not have default values (Python has)
- achieve similar behavior by overloading the method
- with different sets of parameters,
  - where some parameters have default values.

```java
public class DefaultValueExample {
    public void printMessage(String message, int count) {
        for (int i = 0; i < count; i++) {
            System.out.println(message);
        }
    }
    public void printMessage(String message) {
        printMessage(message, 1);
    }
    public void printMessage() {
        printMessage("Default message");
    }
    public static void main(String[] args) {
        DefaultValueExample obj = new DefaultValueExample();
        obj.printMessage();                     // Output: Default message
        obj.printMessage("Hello");              // Output: Hello
        obj.printMessage("Hi", 3);              // Output: Hi Hi Hi
    } // main()
} // class
```

Summary

- Inheritance: Child `extends` Parent
- Access modifier - `public`, `protected`, `private`, `[default]`
  - Package access
- Method overriding - child class method override parent method
  - Dynamic binding
- Method overloading - different method signature